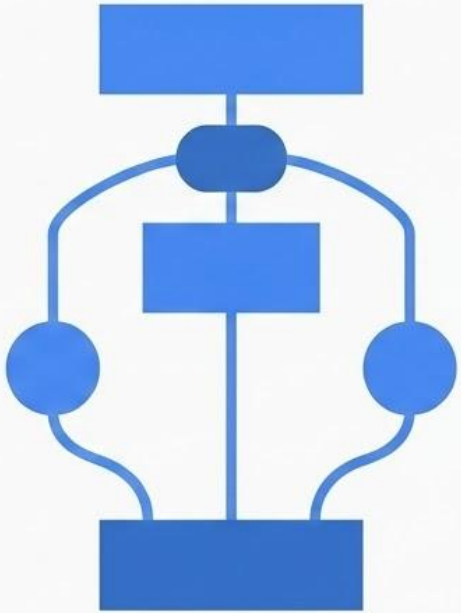
Tip-of-the-Tongue Retrieval: A Multi-Stage Hybrid Retrieval Pipeline with a Single-Turn LLM-Based Query Reformulation



Presenter: Utshab Kumar Ghosh

Debayan Mukhopadhyay
University of Calcutta
debayan.mukherjee14@gmail.com

Utshab Kumar Ghosh
Missouri University of Science and Technology
u.ghosh@mst.edu

Shubham Chatterjee
Missouri University of Science and Technology
shubham.chatterjee@mst.edu

Problem Statement

- Tip-of-the-Tongue (ToT) retrieval addresses the challenge of finding information from fragmented, distorted, or incomplete memories.
- A fundamental disconnect exists between a user's internal, often verbose and uncertain, description and the external identifiers needed to locate an item.
- Traditional semantic similarity and even advanced LLMs often fail on difficult ToT queries due to uncertainty and the risk of hallucination.
- The field is evolving from simple pattern matching to agentic reasoning, where systems must deduce and reconstruct memories, not just rank documents.
- This persistent problem causes significant user frustration and cognitive load, as evidenced by millions of unresolved queries in online communities like Reddit.

Original Methodology and Analysis

As submitted for auto-evaluation on EvalBase before the TREC-TOT 2025 deadline on September 10, 2025 (AOE)

The Initial Hypothesis (Pipeline V1): Complexity & Fusion

Architecture: A complex, 5-stage hierarchical pipeline designed to maximize signal through aggressive fusion.

The Workflow:

- **Stage 1 (Sparse):** Parallel retrieval using BM25, DPH, and TF-IDF on LLaMA-rewritten queries.
- **Stage 2 (Fusion):** Reciprocal Rank Fusion (k=60) to merge the three sparse lists.
- **Stage 3 (Dense):** 3-model ensemble (MiniLM, MPNet, Multi-QA) using weighted fusion (70% Semantic / 30% Sparse).
- **Stage 4 (LTR): LightGBM Learning-to-Rank** model trained on a 6-dimensional feature vector (sparse + dense scores).
- **Stage 5 (Reranking):** Late-interaction reranking (ColBERT-style) on the top 1000 candidates.

Our initial hypothesis was that 'more is better.' We built a heavy, 5-stage pipeline utilizing LightGBM, multiple dense retrievers, and complex Z-score normalization. We thought that by fusing every possible signal, BM25, DPH, MPNet etc. we could brute-force our way to high performance.

V1 Results: The Complexity Trap

Evaluation: Tested on the same held-out test set as the final pipeline.

The Result: Despite the architectural complexity, the pipeline failed to capture relevant documents.

- **Recall@1000: 0.3204** (The "Recall Ceiling" was too low).
- **Success@10: 0.1009** (Only ~10% of queries found a relevant doc in the top 10).
- **nDCG@10: 0.0658** (Ranking quality was negligible).

Post-Mortem: Why Did V1 Fail?

Diagnosis 1: Signal Dilution

- Aggressive fusion (RRF + Weighted + Z-Score) diluted the strong signals with weak ones.

Diagnosis 2: The Recall Bottleneck

- Recall@1000 peaked at **0.32**. The downstream rerankers (LightGBM/ColBERT) were starving for relevant candidates. You cannot rerank what you haven't retrieved.

The Pivot:

- Shift from "Feature Engineering" (LTR) to **"Intent Engineering"** (Reformulation).
- Simplify the pipeline to focus on **High Recall First**, then **High Precision**.
- *Result:* This failure paved the way for the **Reformulation-First Architecture** (Pipeline V2).

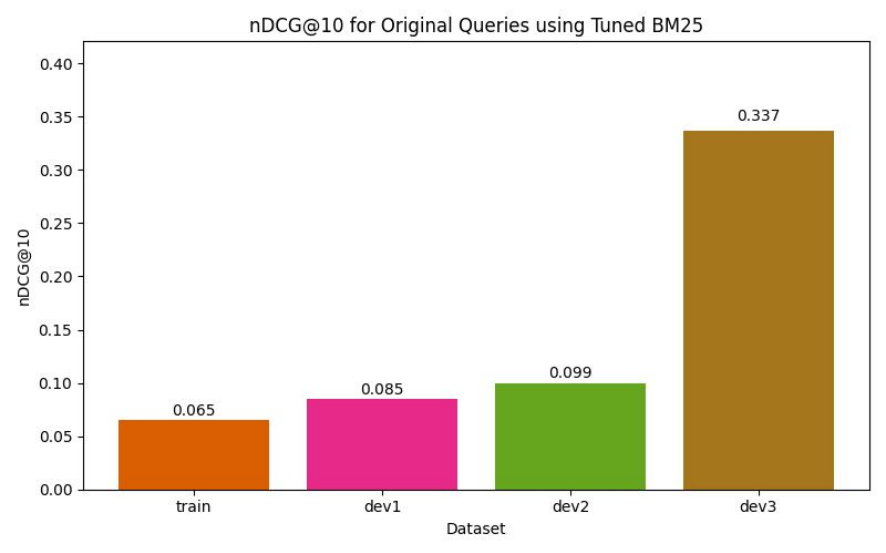
Revised Methodology and Analysis

We started working on Pipeline V2, right after the submission deadline, using the dataset splits and QRELs already available on the Track's website. We were able to validate our improved pipeline using the held out test set, when the track organizers, emailed us the test set QRELs along with the evaluation files of our original pipeline

The Challenge: Tip-of-the-Tongue (ToT) Bottleneck

The Problem: The original queries are riddled with ToT artifacts, diluting the signal of key terms.

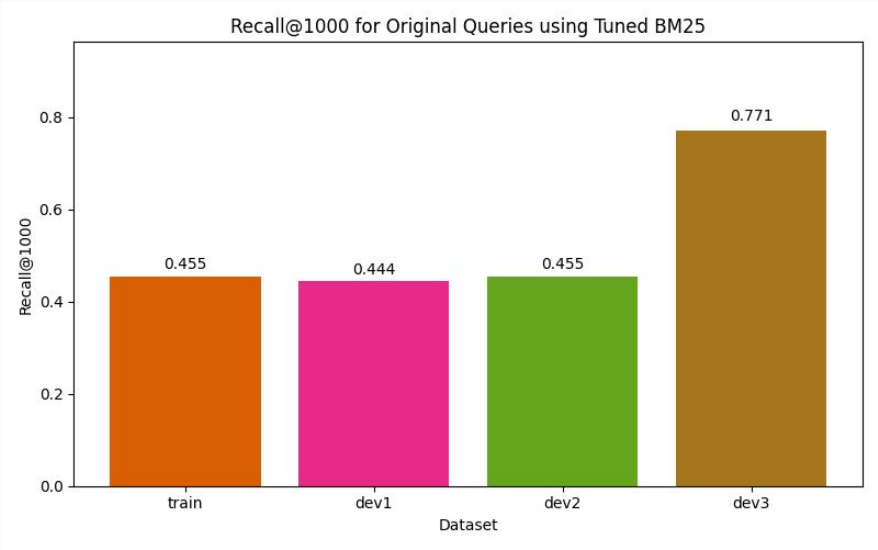
The Consequence: Sparse retrievers (e.g., BM25), even when well-tuned, fail to retrieve relevant documents due to this noise.



The Challenge: Tip-of-the-Tongue (ToT) Bottleneck (contd.)

The Root Cause: The primary bottleneck is Intent Understanding, not ranking.

Strategy: Without extracting stronger signals via Query Reformulation, the pipeline suffers from "Garbage In, Garbage Out" (GIGO).



Query Reformulation Flow

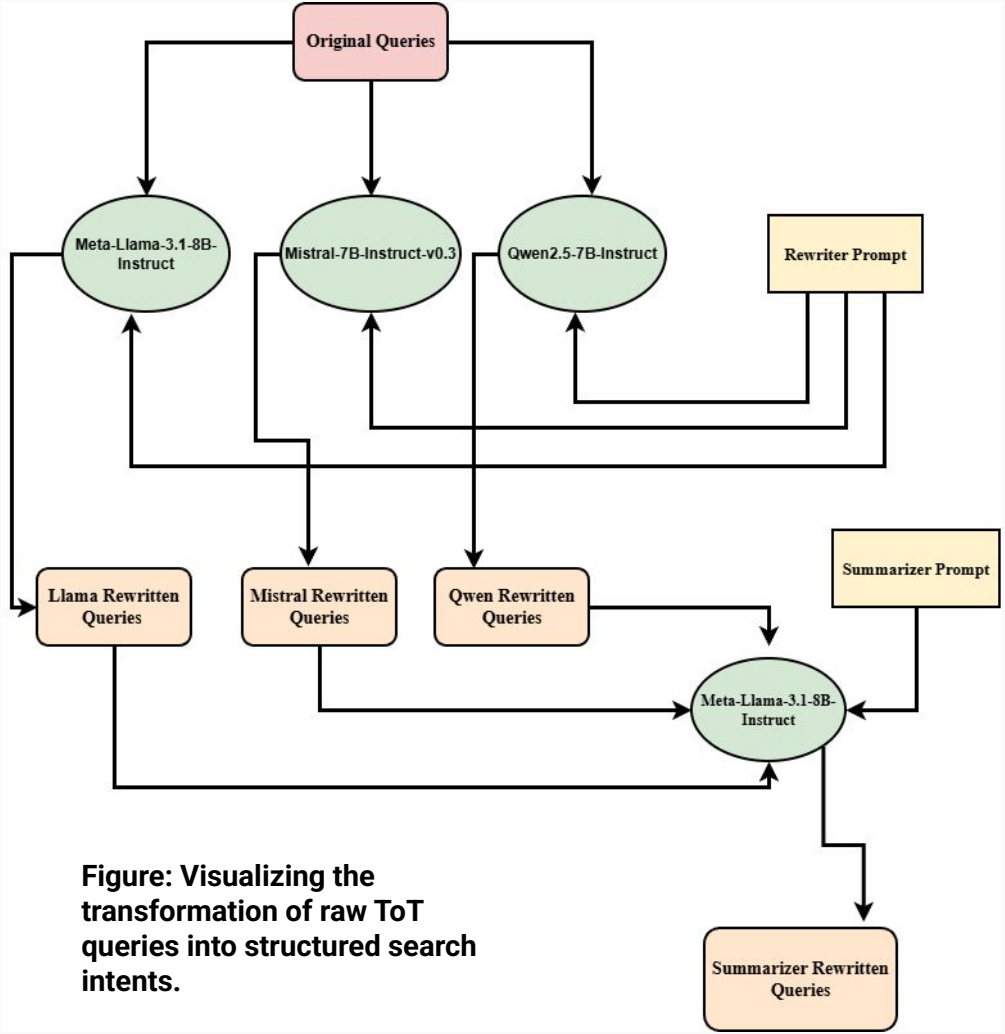


Figure: Visualizing the transformation of raw ToT queries into structured search intents.

Re-writer Prompt Design

Figure: This prompt forces the models to act as strict filters, distilling the user's vague recollection into a structured search query

You are a query rewriter for a search engine. Your task is to rewrite a complex, verbose, tip-of-the-tongue description into a simple, keyword-focused search query.

Guidelines:

1. Identify the core entity type (e.g., movie, book, song) if implied.
2. Extract key details: plot points, character names, actors, famous scenes, or unique identifiers.
3. Remove conversational filler ("I think it was...", "It might be...", "I remember seeing...").
4. Remove negative constraints or uncertainty unless crucial ("Not sure if...").
5. Formulate a concise query that a standard search engine (like Google or BM25) would understand.
6. Output ONLY the rewritten query text. Do not output any explanations.

Query Consensus via Summarization Strategy

Hypothesis: While individual rewriters offer distinct perspectives, a **consensus view** provides the most robust query representation.

The Solution: Introduced a **Summarizer** stage using **Meta-Llama-3.1-8B-Instruct**.

The Mechanism:

- **Input:** Receives the three rewritten query variations.
- **Process:** Fuses them into a single, high-quality query.

Key Advantage:

- **Preserves Signal:** Retains the most salient keywords identified across the ensemble.
- **Reduces Noise:** Filters out hallucinations and idiosyncratic errors found in individual models.

Summarizer Prompt Design

Figure: Prompt instructions for synthesizing a consensus query by fusing rewriter outputs and filtering hallucinations

You are an expert search query optimizer. You will be given three different rewrites of the same original vague query. Your goal is to synthesize the BEST possible search query by combining the unique information from all three sources.

Instructions:

1. Analyze the three rewrites for common keywords and unique, high-value details (e.g., names, specific actions).
2. Create a single, consolidated query that captures the maximum amount of relevant information.
3. Keep it keyword-rich and concise.
4. Output ONLY the final summarized query.

Reformulation Impact: Ranking Quality (nDCG@10)

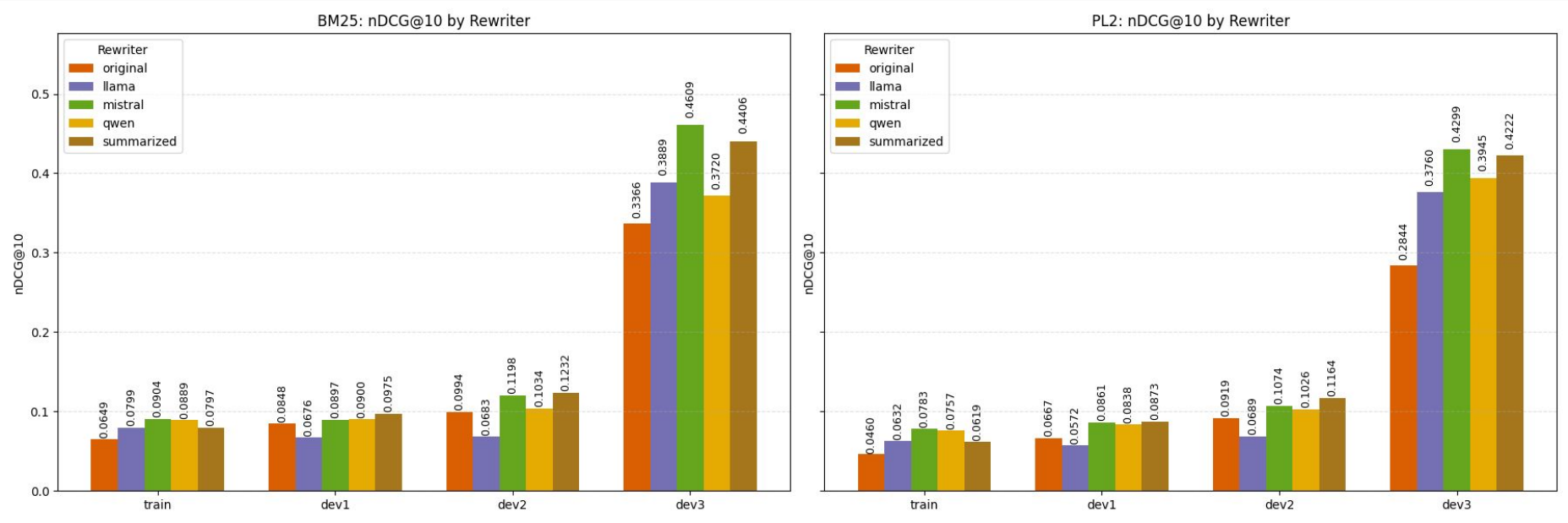


Figure: Comparative analysis of nDCG@10 scores across dataset splits. The significant lift over the baseline confirms that reformulation effectively restores ranking signal in ToT queries.

Reformulation Impact: Retrieval Coverage (Recall@1000)

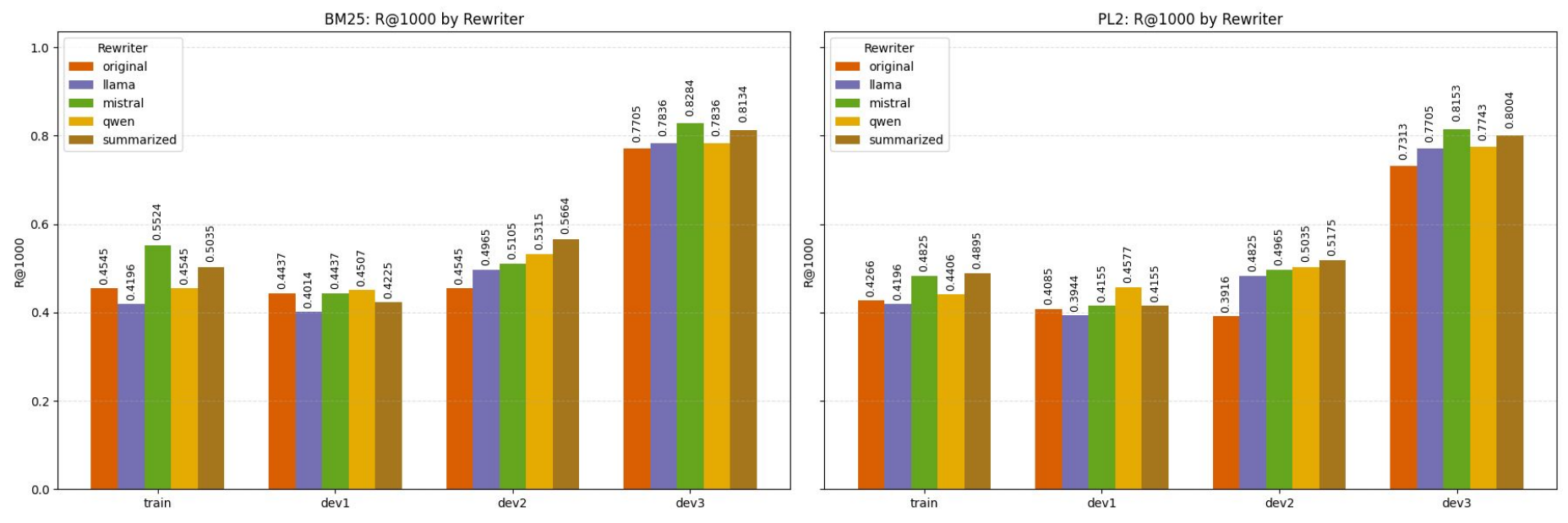


Figure: Impact of reformulation on Recall@1000. High recall at this stage is critical to ensure the initial candidate pool contains the relevant documents for downstream re-ranking.

Optimal Strategy Selection: Performance vs. Cost

Performance Evaluation:

- Tested across dataset splits using tuned BM25 and PL2 models.
- **Observation: Mistral** rewritten queries consistently achieved the highest Recall@1000 and nDCG@10 scores.
- **Runner-up:** The Summarized (Consensus) approach ranked second.

Strategic Decision:

- We proceeded downstream using **Mistral Rewritten Queries** exclusively.

Rationale (Efficiency):

- The Summarizer is computationally expensive, requiring **4 LLM prompts per query** (3 rewriters + 1 summarizer).
- Mistral offers superior metrics with significantly lower latency and compute cost.

Optimal Strategy Selection: Performance vs. Cost

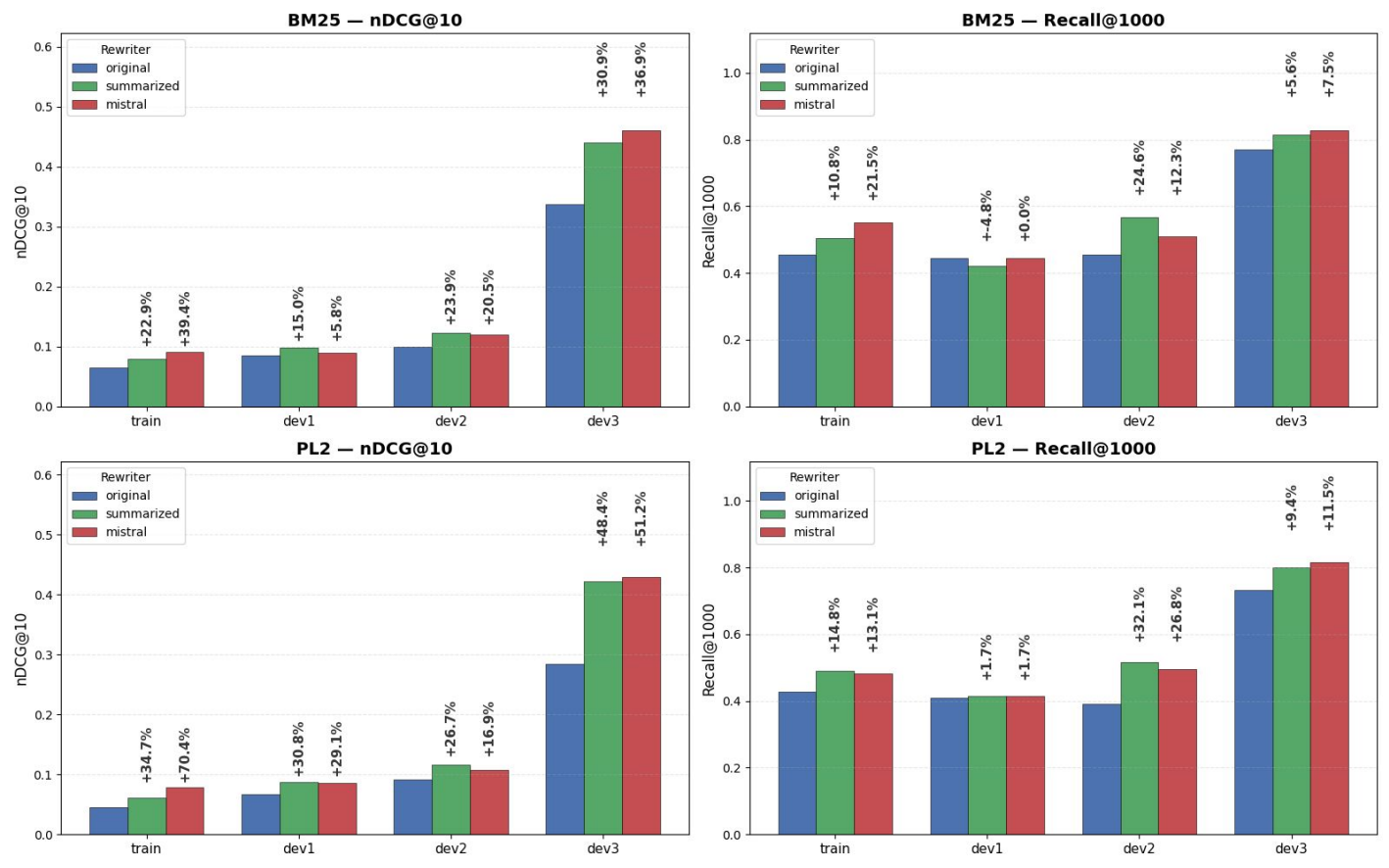


Figure: Quantifying the "Reformulation Lift": Mistral-rewritten queries show massive gains over the original baseline across both sparse (BM25 and PL2) retrieval models.

Four Stage Retrieval Pipeline to Iteratively Refine Results

STAGE 01

Sparse Retrieval: "Cheap" and fast initial candidate generation from the full document corpus. Focusing on surface-level lexical matching to cast a wide net.

STAGE 02

Bi-encoder Ensemble: Semantic re-ranking of the initial pool to capture context and meaning that keyword matching missed. Addressing the "vocabulary mismatch" problem.

STAGE 03

Cross-encoder Re-ranker: Evaluating the specific query-document pairs with high precision to filter out non-relevant documents before the final stage.

STAGE 04

List-wise LLM Re-ranker: Utilizing specific LLM capabilities to resolve final ambiguities and order the top results based on world knowledge and complex intent understanding.

Candidate Set (K) is optimally pruned at every stage to reduce computational expense

Stage 1: Sparse Retrieval Baseline and Parameter Optimization Strategy

Objective: Establish a high-recall "safety net" to maximize candidate coverage.

Models:

- **Okapi BM25:** Standard probabilistic baseline.
- **PL2 (Divergence from Randomness):** Captures term distribution divergence.

Optimization Strategy: Sequential Adaptive Grid Search

- **Optimization Objective:** Maximize nDCG@10 (Ranking Quality).
- **Flow:** Employed a multi-stage tuning approach: train → dev1 → dev2 (*We did not use dev3 for tuning parameters at any stage, as it served as our pseudo-test set*).
- **Mechanism:** Progressed from a coarse **Global Search** (Train) to a fine-grained **Local Refinement** (Dev sets) around optimal parameters to prevent overfitting.
- *(Full algorithmic details and search spaces available in our Notebook Paper Section 3.4.1)*

Stage 1: Sparse Retrieval Performance (BM25 vs. PL2)

rewriter	model	params	ndcg_10	R@1000
mistral	BM25	{'k1': 0.01, 'b': 0.55}	0.4609	0.8284
mistral	PL2	{'c': 2.5}	0.4299	0.8153
summarized	BM25	{'k1': 0.01, 'b': 0.55}	0.4406	0.8134
summarized	PL2	{'c': 3.5}	0.4222	0.8004
llama	BM25	{'k1': 0.01, 'b': 0.6}	0.3889	0.7836
qwen	BM25	{'k1': 0.01, 'b': 0.85}	0.3720	0.7836
qwen	PL2	{'c': 2.5}	0.3945	0.7743
llama	PL2	{'c': 3.5}	0.3760	0.7705
original	BM25	{'k1': 0.01, 'b': 0.75}	0.3366	0.7705
original	PL2	{'c': 1.76}	0.2844	0.7313

Table: Mistral-rewritten queries outperform Original and Summarized queries on both retrieval models.

Stage 1: Sparse Retrieval Performance (BM25 vs. PL2) (contd.)

Metric: nDCG@10 (Proxy for Intent Alignment).

Optimized Parameters:

- The Adaptive Grid Search converged on distinct "sweet spots" for each rewriter.
- **Observation:** Mistral-rewritten queries consistently yielded the highest nDCG scores across both BM25 and PL2.

Correlation Note:

- While we optimized for nDCG@10, we observed a near-perfect correlation with Recall@1000. Optimizing for top-tier ranking inherently ensured broad retrieval coverage.

Stage 1: Why we dropped the idea of Sparse RRF?

Initial Hypothesis:

- Combine BM25 and PL2 outputs using **Reciprocal Rank Fusion (RRF)** to maximize recall before the Dense Stage.

Empirical Finding:

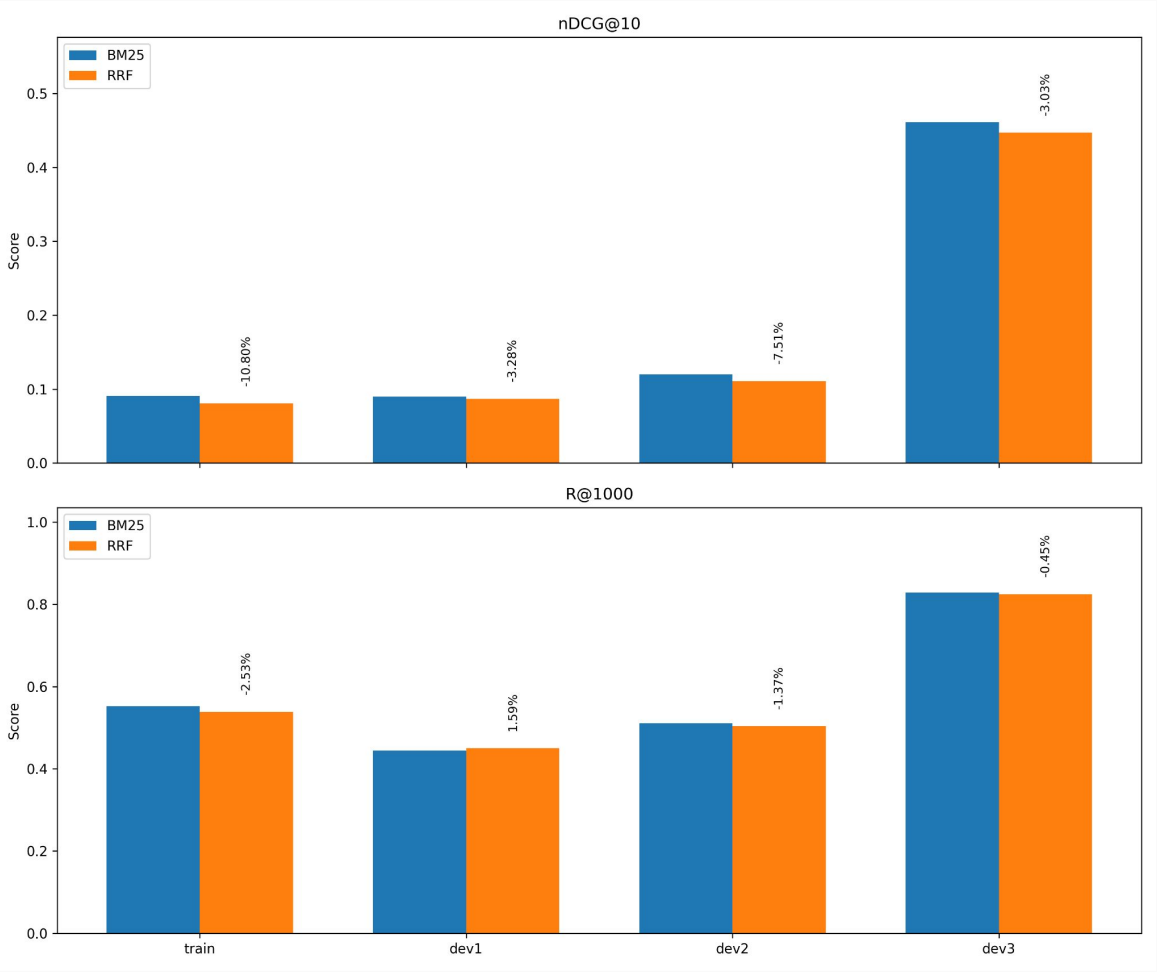
- RRF added computational overhead (running two retrievers + fusion logic) with negligible performance gain
- Infact, the scores of the better performing BM25 was getting pulled down by the RRF as it was incorporating PL2's ranked list

Decision:

- **Drop PL2 and RRF.**
- Proceed to the Dense Stage using **only BM25 candidates** derived from Mistral queries.

Benefit:

- Reduces Stage 1 latency by **50%** without compromising candidate quality.



Stage 2: Bi-encoder Ensemble: Bridging the Semantic Gap

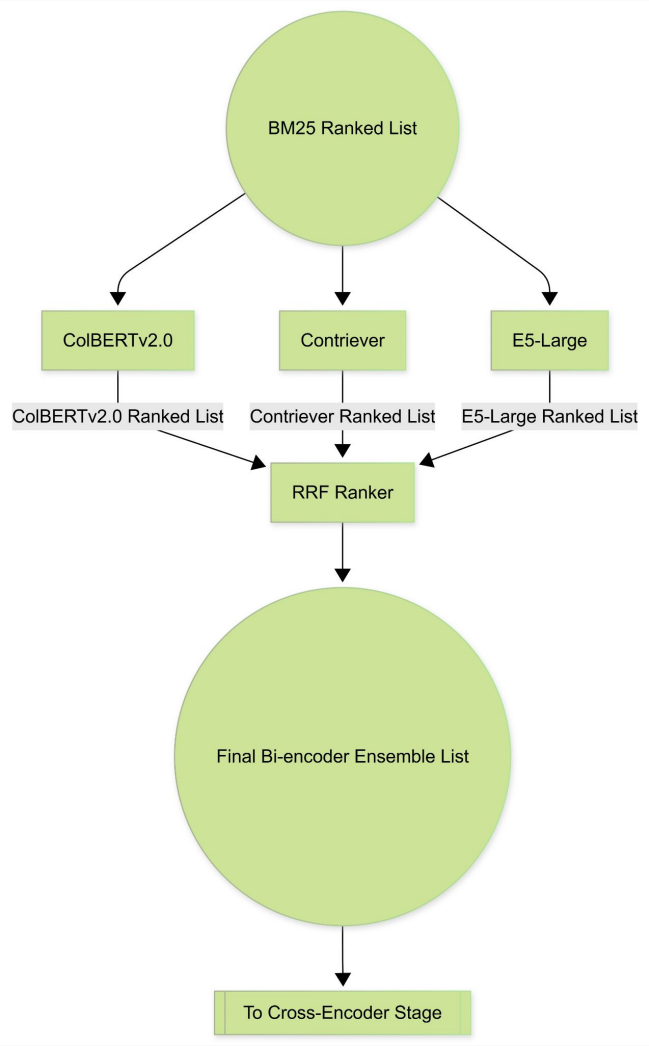
Objective: Address the "Vocabulary Mismatch" in ToT queries by capturing deep semantic signals (High-Recall Filter).

The Ensemble Approach: Implemented a parallel architecture of three distinct bi-encoders:

- **ColBERTv2.0:** Late-interaction model; retains token-level granularity for verbose queries.
- **Contriever:** Contrastive learning model; excellent zero-shot robustness.
- **E5-Large:** Weakly supervised; provides strong semantic matching.

Fusion Mechanism:

- Outputs combined via **Reciprocal Rank Fusion (RRF)** (k=60).
- Result: A single, robust candidate list leveraging the strengths of all three architectures.



Stage 2: Optimization Strategy: Robust Candidate Depth (K_dense)

The Challenge: Determining the optimal number of documents (K_dense) to pass to the next stage.

Metric Shift:

- Unlike Stage 1 (optimized for nDCG), Stage 2 is optimized purely for **Recall**.
- *Goal:* Ensure the relevant document survives the filter.

Robust Aggregate Tuning:

- Avoids overfitting to a single split by penalizing instability.
- **Formula:** $S_K = \mu(R@K) - \alpha \cdot \sigma(R@K)$
- Selects a depth that performs consistently across Train, Dev1, and Dev2.

Efficiency: Used "**Virtual Tuning**" (running inference once for Top-5k and slicing results) to rapidly test thousands of (K_dense) values.

Stage 2: Performance Analysis: Ranking Degradation vs. Recall Stability

Observation:

- Performance plateaued decisively across all three dense models.
- Increasing candidate depth (K) from **1,000 to 5,000** yielded **zero improvement** in Robust Score or Mean Recall.

The "Recall Ceiling" Phenomenon:

- The maximum recoverable recall is strictly determined by the top-1000 documents retrieved by the previous (Sparse) stage.
- The Dense stage cannot "find" documents that the Sparse stage did not retrieve.

Decision:

- Selected **K = 1000** as the optimal cutoff.
- *Rationale:* Fetching candidates beyond this point incurs computational cost with absolutely no potential gain in coverage.

Depth (K)	Mean Recall	Std Dev	Robust Score
500	0.4134	0.0335	0.3967
1000	**0.5022**	**0.0448**	**0.4798**
1500	0.5022	0.0448	0.4798
...	...	...	...
5000	0.5022	0.0448	0.4798

Table: Results of Robust Candidate Depth Tuning for ColBERT.The performance plateaus at K=1000, indicating that deeper sparse retrieval yields no added relevant documents.

Stage 2: Performance Analysis: Ranking Degradation vs. Recall Stability

The Phenomenon:

- Comparison of Input (Mistral BM25) vs. Output (Dense Fusion) reveals a critical divergence.

1. Ranking Degradation (nDCG@10 ↓18%) on Pseudo-Test Set:

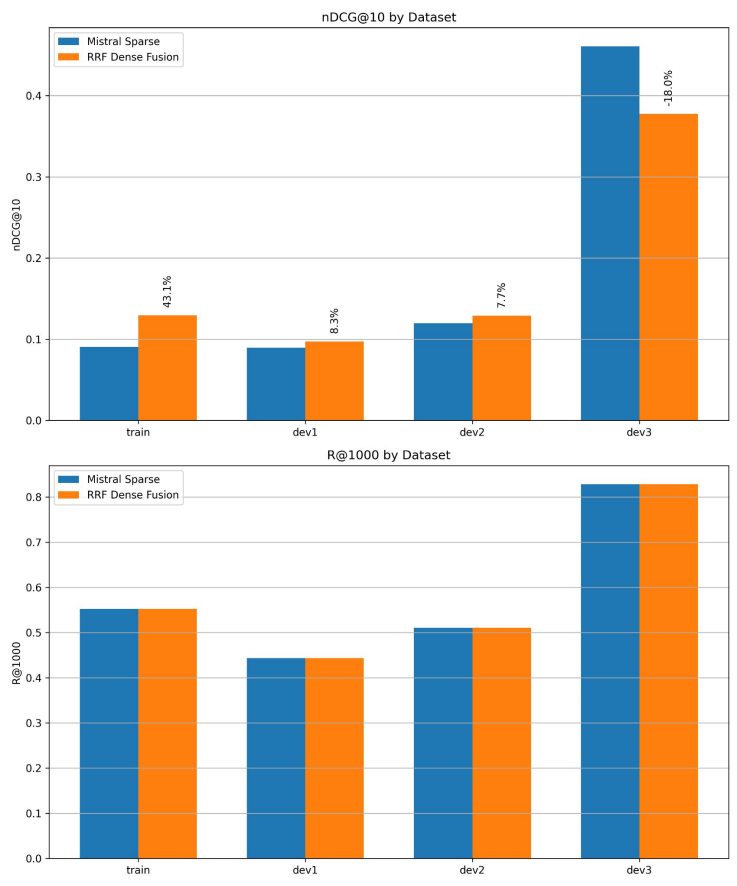
- *Cause:* Mistral queries are "keyword-optimized" for BM25.
- Dense models, trained on natural semantic matching, likely penalized this "keyword-stuffing," pushing exact matches down in favor of broader semantic matches.

2. Recall Stability (Recall@1000 Constant):

- **Recall remained identical at 0.8284.**
- *Implication:* No relevant documents were lost; they were merely displaced from the top 10 to lower positions (ranks 11–100).

Strategic Decision:

- **Proceed with Dense Fusion.**
- We rely on the subsequent **Cross-Encoder** stage to repair the ranking order. The priority here was *preserving* the candidate pool, which was achieved.



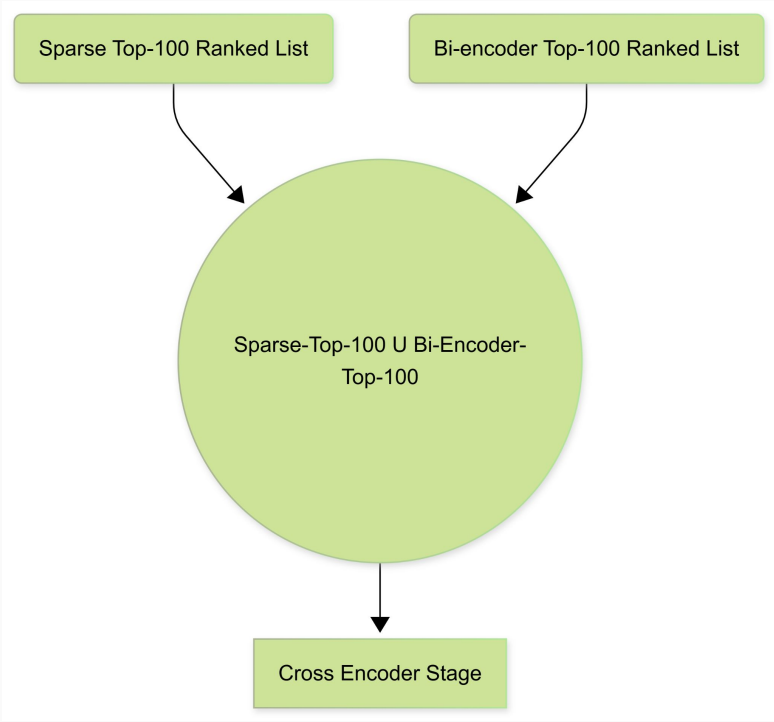
Stage 3: Cross Encoder Re-Ranker

The Problem: Dense retrieval introduced "Semantic Drift"—prioritizing broad topics over specific keyword constraints.

The Solution: Deployed **monoT5-large** as a Cross-Encoder to perform fine-grained relevance assessment (processing query and document simultaneously).

Methodology: Hybrid Input Pooling

- To maximize coverage, we didn't just re-rank the Dense output.
- **The Union Pool:** Constructed a candidate list by merging:
  - **Top-100** from Sparse BM25 (Exact Match Signal)
  - **Top-100** from Dense RRF (Semantic Signal)
- ***Result:*** Ensures the Cross-Encoder has access to both precise keyword matches and broad semantic candidates.



Stage 3: Performance Analysis of the Cross Encoder Re-Ranker

"Phoenix-like" Recovery:

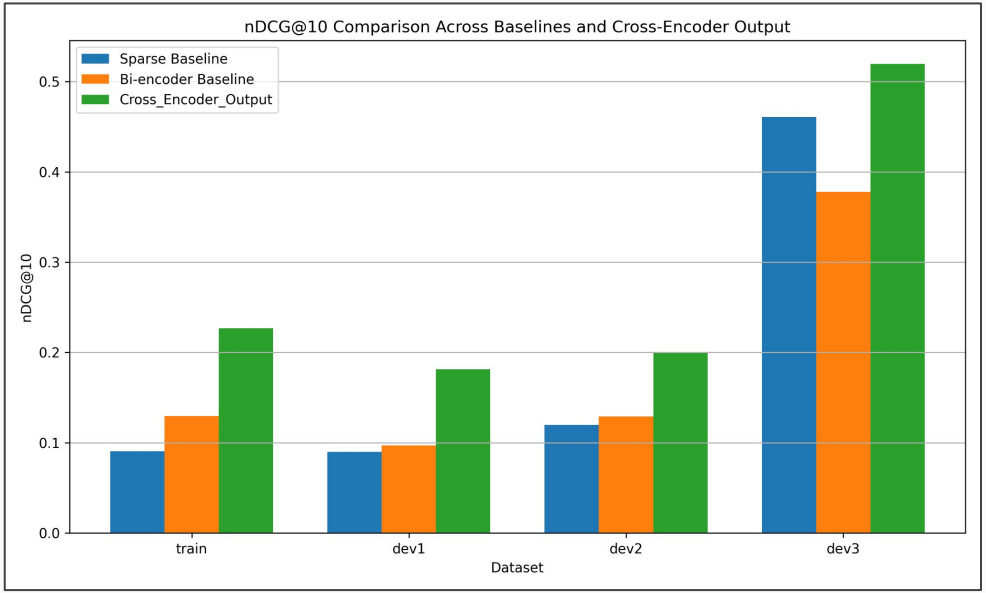
- The Cross-Encoder repaired the 18% performance drop incurred during the Dense stage and significantly outperformed the initial Sparse baseline.

Key Metrics:

- **Success@10:** Boosted from **0.54** → **0.65**.
- **Mechanism:** Successfully identified relevant documents buried deep in the list (ranks 11+) and elevated them to the top.

The Trade-off:

- Total Recall dropped from **0.828** → **0.733**.
- *Analysis:* We sacrificed "long-tail recall" (documents ranked >300) to secure massive improvements in top-tier precision.



Stage 3: Optimizing Re-ranking Depth for Latency

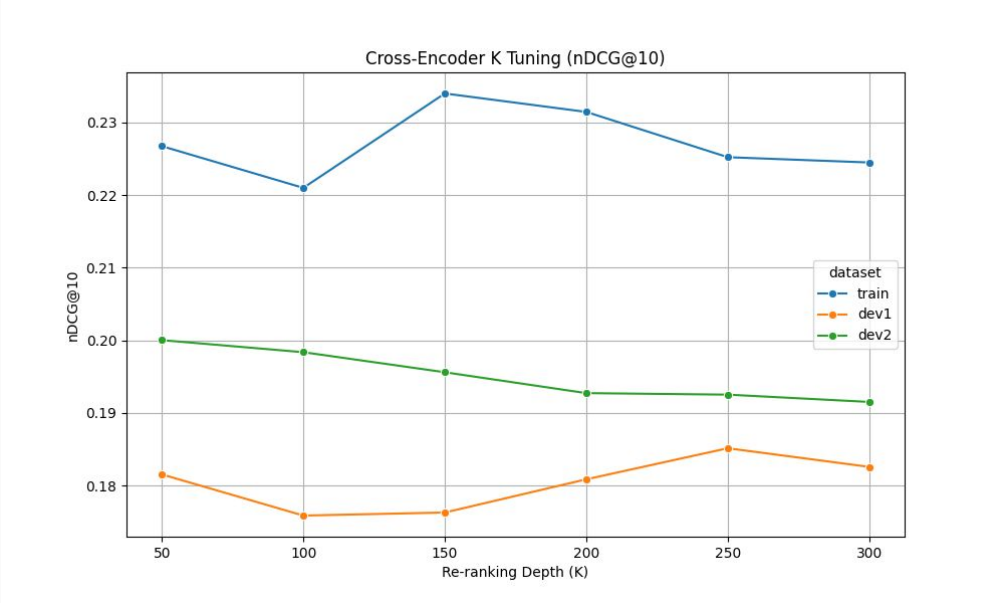
Constraint: Cross-Encoders are computationally expensive; minimizing the candidate depth (K) is crucial for latency.

Tuning Observation:

- Robust tuning sweeps revealed that the "relevance signal" is concentrated in the very top portion of the input list.
- Re-ranking deeper candidates (Depth 100–300) introduced more **noise** than relevance, actually diluting the robust score.

Final Configuration:

- **Set K = 50**
- *Impact:* Maximizes ranking quality while ensuring maximum efficiency for the final LLM stage.



Stage 4: List-wise LLM Re-Ranker

- **Model: Qwen2.5-72B-Instruct** (4-bit quantized).
- **Strategy: Listwise Re-ranking**.
  - Unlike point-wise scoring, the LLM evaluates a batch of candidates *simultaneously*.
  - Allows the model to compare candidates directly against each other to resolve subtle ambiguities.
- **Context Management:**
  - **Truncation:** Documents truncated to the first **500 characters** (approx. Wikipedia Introduction length).
- **Why Truncate?**
  - **"Lost in the Middle" Phenomenon:** LLMs focus best on the start/end. Relevant ToT signals (plot summaries) are usually in the intro.
  - **Cost/Latency:** Full text (2k tokens) for 50 candidates would create 100k+ token inputs, exploding latency from seconds to minutes.

Stage 4: Optimizing Efficiency: Joint Parameter Tuning

Objective: Balance maximum precision with computational cost.

Method: Joint Grid Search optimizing two variables:

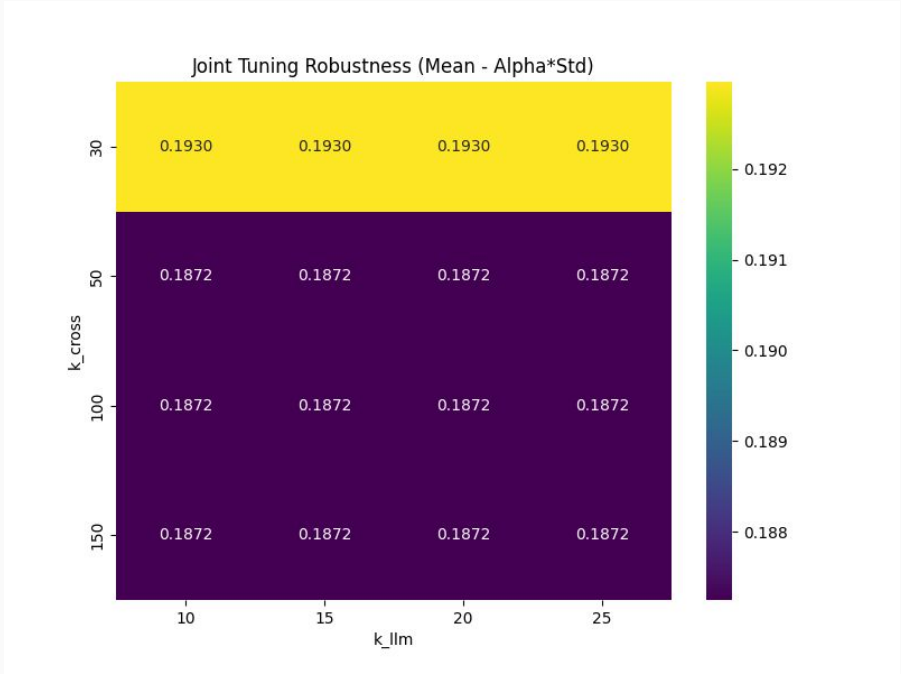
- K_cross: Input window size passed from Cross-Encoder.
- K_llm: Output depth of the re-ranked list.

Key Finding:

- Performance saturates quickly. Smaller windows (K_cross=30) matched or outperformed larger ones (K_cross=50+).
- **Conclusion:** Relevant signals are concentrated at the very top. Processing ranks 31–100 introduces **noise**, not signal.

Final Configuration:

- **K_cross = 30** (Input)
- **K_llm = 10** (Output)



Stage 4: Final Results: Generalization vs. Overfitting

Blind Test Set (Dev3) Results:

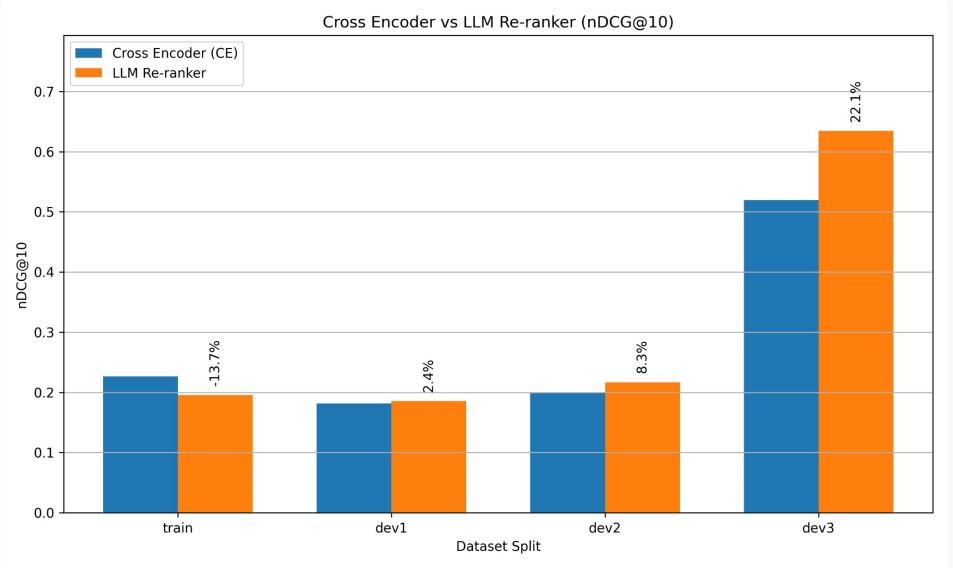
- **nDCG@10: 0.6347**
- **MRR Boost: +29.2%** over Cross-Encoder.
- *Significance:* The LLM excels at moving the ground truth to the precise Rank-1 position.

The "Regression" Anomaly:

- **Observation:** While Test performance soared, **Training set performance dropped (-13.7%)**.

Explanation:

- **Pipeline Overfitting:** Previous stages (Sparse/Dense/Cross) were heavily tuned on the Train set, likely overfitting to specific patterns.
- **LLM Robustness:** The LLM acts as a **Zero-Shot Generalist**. It ignores dataset-specific artifacts and re-ranks based on broad semantic reasoning.
- *Conclusion:* The LLM corrects the overfitting, resulting in a slightly lower Train score but massive gains on unseen data.



Metric	Cross-Encoder	LLM Reranker	Gain
nDCG@10	0.5196	**0.6347**	+22.1%
Success@10	0.6549	**0.6828**	+4.3%
MRR	0.4808	**0.6213**	+29.2%

Table: Performance comparison between the Cross Encoder(Input) and LLM Re-ranker(Output) on the pseudo-test set(dev3).

Final Pipeline Validation

Pipeline Stage	nDCG@10	Success@10	Recall@1000	Gain (nDCG)
Sparse Baseline (BM25)	0.1738	0.2476	0.6302	--
Dense Fusion (RRF)	0.2338	0.3457	0.6302	+34.5%
Cross-Encoder (MonoT5)	0.311	0.4244	0.5756	+78.9%
LLM Reranker (Qwen)	0.3706	0.455	0.5756	+113.2%

Test Set Characteristics:

- Evaluated on a strictly **held-out test set**.
- **Challenge Level:** Significantly higher than development sets.
- **Features:** Characterized by lower absolute recall baselines and high query ambiguity (hard ToT cases).

Performance Impact:

- Achieved a **+113.2% gain** in nDCG@10 over the sparse baseline.
- **Success@10** nearly doubled (0.24 → 0.45).

Validation:

- Results confirm that a single-stage retriever cannot handle ToT tasks. A cascaded approach is essential.

Impact of Architectural Evolution (V1 vs. V2)

Metric	Pipeline V1	Pipeline V2	Performance Lift
nDCG@10	0.0658	0.3706	+463.2%
Success@10	0.1009	0.4550	+350.9%
Recall@1000	0.3204	0.5756	+79.6%

The Paradigm Shift:

- **Pipeline V1 (Failed):** Focused on *Feature Engineering* (Learning-to-Rank, aggressive fusion) → Failed due to low recall and signal dilution.
- **Pipeline V2 (Final):** Focused on *Intent Engineering* (Reformulation) and *Semantic Interaction* (Cross-Encoder/LLM) → Succeeded by fixing the input and using deeper models.

Quantitative Lift:

- **Ranking Quality (nDCG@10): +463%** improvement. The system went from "random guessing" to state-of-the-art ranking.
- **User Success (Success@10): +351%** increase. The probability of a user finding their answer in the top 10 rose from **10%** to **45%**.

Conclusion:

- Complexity is not enough. For ambiguous ToT queries, **reformulation is non-negotiable**.

To Conclude:

Rewrite First Then Retrieve

- ToT query artifacts are the primary performance bottleneck.
- LLM-based query rewriting is not an optional preprocessing step, it is the foundational requirement for turning a vague recollection into a machine-readable intent.

Cascade From Recall to Precision

- A multi-stage architecture is essential.
- Use a high-recall foundation to cast a wide net, then apply computationally expensive, high-precision models like Cross-Encoders and LLMs to refine a much smaller, high-value candidate set.

Deploy LLMs as Last Stage Semantic Reasoners

- The final precision gains come from large language models (72B+) acting as a listwise re-ranker.
- Their extensive world knowledge allows them to resolve ambiguities that smaller, specialized models cannot.

THANK YOU